Módulo de Autenticación y Control de Acceso

El módulo desarrollado corresponde al componente de autenticación y control de acceso del sistema **FlotaSegura**. Su objetivo es validar la identidad de los usuarios antes de permitir el acceso a las distintas funcionalidades del sistema y controlar posteriormente qué operaciones puede realizar cada usuario según su rol.

El desarrollo se implementó en el archivo auth.py, incorporando mecanismos de inicio de sesión, almacenamiento seguro de contraseñas mediante hash, generación y validación de tokens y control de acceso basado en roles (RBAC).

Esta implementación se relaciona directamente con el requerimiento funcional definido previamente para FlotaSegura, donde se establece que el sistema debe exigir inicio de sesión mediante credenciales personales y habilitar únicamente las funciones correspondientes al rol del usuario: **Conductor, Jefe de Flota o Gerente**. Informe_UA2_FlotaSegura_corregi…

Además, el módulo responde al requerimiento no funcional de seguridad del sistema, el cual establece el uso de control de acceso basado en roles y el almacenamiento de contraseñas mediante hash.

Responsable: Yerko Morales

Archivo desarrollado: Auth.py

Archivo de pruebas: test_auth.py

Repositorio: ypmorb/FlotaSegura: Prototipo del sistema FlotaSegura - Ingenieria de Software

1.Evidencias



Repositorio del prototipo FlotaSegura con archivos correspondientes al módulo de autenticación y pruebas.

2. Funcionamiento técnico del módulo auth.py

El archivo auth.py concentra la lógica principal de autenticación y autorización del prototipo FlotaSegura. Su función es comprobar la identidad del usuario, proteger sus credenciales y restringir el acceso a las funcionalidades según el rol asignado.

El módulo implementa cuatro mecanismos principales: almacenamiento seguro de contraseñas mediante hash, validación de credenciales durante el inicio de sesión, generación y validación de tokens de autenticación y control de acceso basado en roles.

Esta solución mantiene coherencia con el diseño previo del sistema, donde el inicio de sesión y el control de acceso por rol forman parte de los requerimientos funcionales y no funcionales de seguridad.

Función	Propósito
hash_password()	Convierte la contraseña en un hash seguro antes de almacenarla.
verify_password()	Compara la contraseña ingresada con el hash almacenado.
login()	Valida usuario y contraseña y genera un token si las credenciales son correctas.
generate_token()	Crea un token asociado al usuario, rol y tiempo de expiración.

Función	Propósito
validate_token()	Comprueba que el token sea válido, no haya sido alterado y no esté vencido.
require_role()	Verifica que el usuario posea uno de los roles autorizados para ejecutar una operación.
registrar_kilometraje()	Simula una operación permitida para el rol Conductor.
gestionar_mantenciones()	Simula una operación restringida al Jefe de Flota.
consultar_reportes()	Permite acceso a Jefe de Flota y Gerente.

3. Protección de contraseñas

Para evitar almacenar contraseñas en texto plano, el módulo utiliza un proceso de hash mediante PBKDF2-HMAC-SHA256. Cada contraseña se combina además con un valor aleatorio denominado salt, lo que dificulta que dos usuarios con la misma contraseña generen exactamente el mismo valor almacenado.

Durante el inicio de sesión, la contraseña ingresada no se compara directamente con el valor original, sino que se vuelve a procesar utilizando el mismo salt y posteriormente se compara con el hash almacenado.

```
68  def hash_password(password: str, salt: str = None) -> Dict[str, str]:
69      """
70      Genera un hash seguro de la contraseña utilizando PBKDF2-HMAC-SHA256.
```

```
98  def verify_password(password: str, salt: str, stored_hash: str) -> bool:
99      """
100     Compara una contraseña ingresada con el hash almacenado.
101     """
```

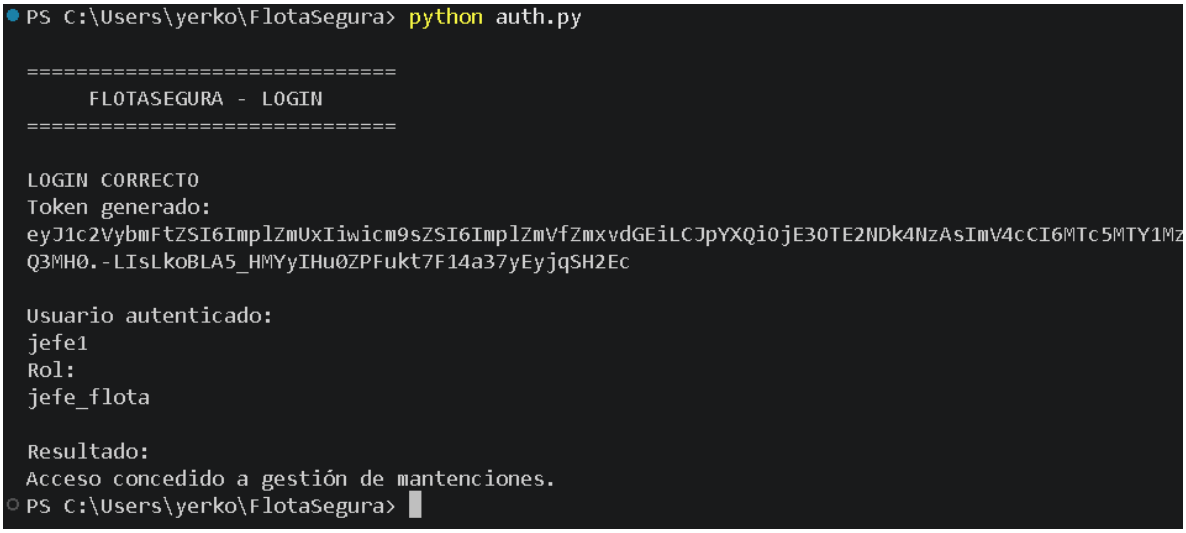
Implementación del mecanismo de hash y verificación de contraseñas en el módulo de autenticación.

4. Inicio de sesión y generación de token

El proceso de inicio de sesión comienza mediante la función login(), la cual recibe el nombre de usuario y la contraseña. Si alguno de los campos se encuentra

vacío, el usuario no existe o la contraseña es incorrecta, el sistema rechaza la autenticación.

Cuando las credenciales son válidas, el sistema genera un token que contiene información del usuario, su rol y un tiempo de expiración. Este token permite identificar al usuario en operaciones posteriores sin volver a solicitar sus credenciales en cada acción.



Ejecución exitosa del módulo de autenticación de FlotaSegura, donde se valida el inicio de sesión, la generación del token y la identificación del rol del usuario.

5. Plan de pruebas del módulo de autenticación

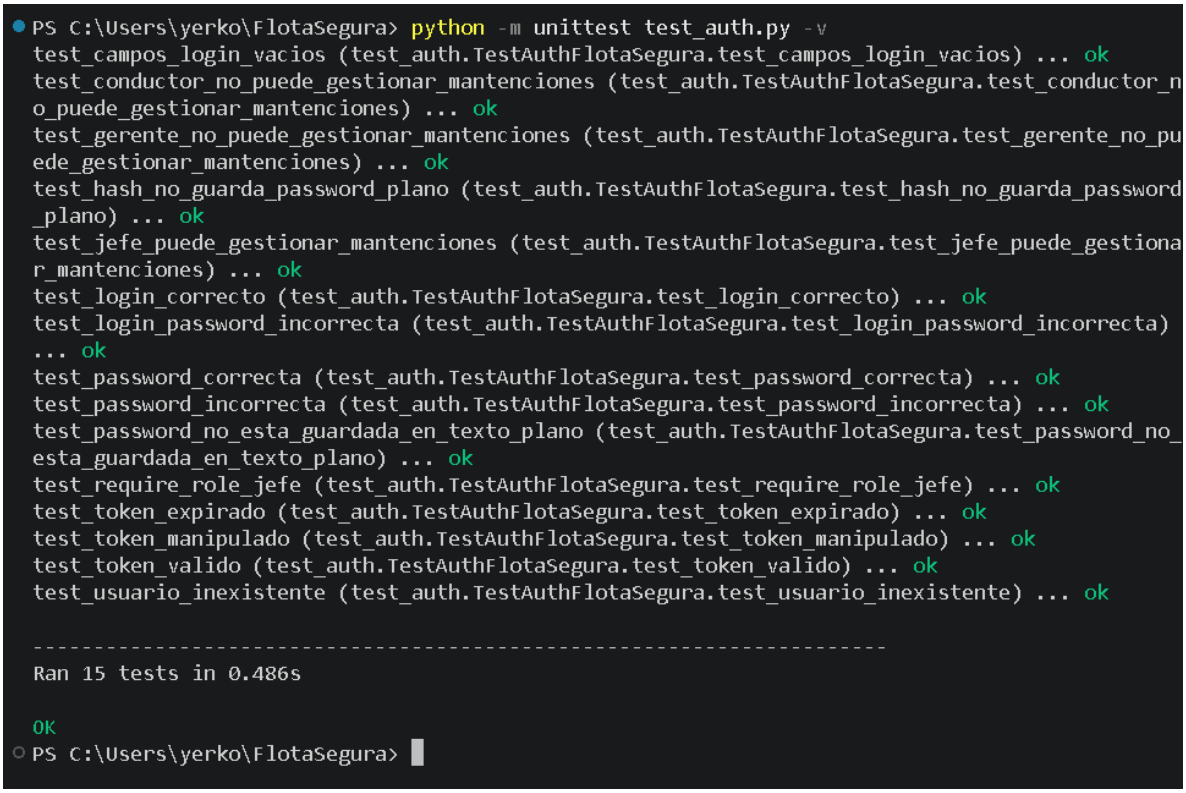
Para verificar el correcto funcionamiento del módulo de autenticación se diseñó un conjunto de pruebas automatizadas mediante unittest en el archivo test_auth.py.

Las pruebas consideran escenarios exitosos y escenarios de error relacionados con autenticación, protección de contraseñas, validación de tokens y control de acceso por roles. El objetivo es comprobar que el sistema no solo permita el acceso a usuarios válidos, sino que también rechace credenciales incorrectas, tokens manipulados o vencidos y operaciones realizadas por usuarios sin autorización.

Estas pruebas se relacionan directamente con el requerimiento de seguridad definido previamente para FlotaSegura, donde se establece que el sistema debe utilizar autenticación personal, control de acceso basado en roles y mecanismos de protección de credenciales.

ID	Caso de prueba	Resultado esperado
CP-01	Verificar que la contraseña no se almacene en texto plano	El hash debe ser diferente de la contraseña original
CP-02	Validar contraseña correcta	La comparación debe retornar verdadero
CP-03	Validar contraseña incorrecta	La comparación debe retornar falso
CP-04	Login con credenciales correctas	Se genera un token válido
CP-05	Login con contraseña incorrecta	El acceso es rechazado
CP-06	Login con usuario inexistente	El acceso es rechazado
CP-07	Login con campos vacíos	El sistema genera un error controlado
CP-08	Validación de token correcto	El sistema identifica usuario y rol
CP-09	Token manipulado	El sistema rechaza el token
CP-10	Token expirado	El sistema rechaza el acceso
CP-11	Conductor intenta gestionar mantenciones	Acceso denegado
CP-12	Jefe de Flota gestiona mantenciones	Acceso permitido
CP-13	Gerente intenta gestionar mantenciones	Acceso denegado
CP-14	Validación directa del rol Jefe de Flota	Rol autorizado correctamente

ID	Caso de prueba	Resultado esperado
CP-15	Verificación de contraseña almacenada	No debe existir la contraseña en texto plano



Ejecución automatizada de los 15 casos de prueba del módulo de autenticación de FlotaSegura, finalizados satisfactoriamente. Elaboración propia.

Los resultados obtenidos muestran que, luego de aplicar las correcciones necesarias, los 15 casos de prueba finalizaron satisfactoriamente. Esto permite verificar el funcionamiento del inicio de sesión, protección de contraseñas, generación y validación de tokens y control de acceso por roles.

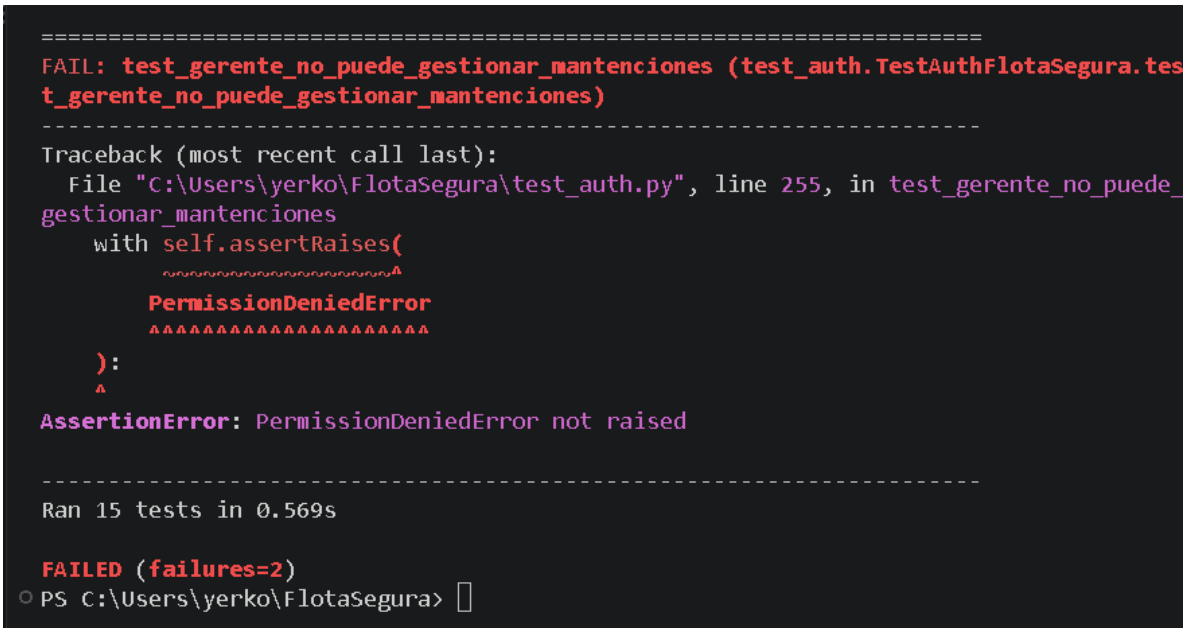
6.Falla detectada durante las pruebas

Durante la ejecución del plan de pruebas se detectó una falla en el control de acceso basado en roles (RBAC). La primera versión del módulo validaba

correctamente el token de autenticación, pero no verificaba si el rol del usuario estaba autorizado para ejecutar una determinada función.

Como consecuencia, usuarios autenticados podían intentar acceder a operaciones que no correspondían a sus permisos. Esta situación fue detectada mediante las pruebas automatizadas test_conductor_no_puede_gestionar_mantenciones y test_gerente_no_puede_gestionar_mantenciones.

La falla era relevante porque contradecía directamente el requerimiento de seguridad definido previamente para FlotaSegura, donde se establece que cada usuario debe acceder únicamente a las funciones correspondientes a su rol.



Resultado de las pruebas automatizadas antes de la corrección, donde se detectan dos fallas asociadas al control de acceso por roles. Elaboración propia.

La ejecución mostró que dos casos de prueba fallaron porque el sistema no estaba rechazando correctamente el acceso de usuarios con roles no autorizados.

7. Análisis de causa

7.1 Causa identificada

El análisis del código permitió determinar que la función require_role() validaba únicamente que el token fuera correcto, pero no comprobaba si el rol contenido en el token pertenecía a la lista de roles permitidos para la operación solicitada.

Por lo tanto, el mecanismo de autenticación funcionaba, pero la autorización estaba incompleta. En términos simples, el sistema podía reconocer quién era el usuario, pero todavía no verificaba adecuadamente qué acciones tenía permitido realizar.

8. Corrección aplicada

Para solucionar la falla se modificó la función require_role(), incorporando una validación explícita del rol del usuario. Después de validar el token, el sistema obtiene el rol asociado y lo compara con los roles autorizados para la funcionalidad solicitada.

Si el rol no se encuentra dentro de los permitidos, el sistema genera una excepción PermissionDeniedError y rechaza la operación. Con esta modificación se implementó correctamente el control de acceso RBAC definido en los requerimientos de seguridad del sistema.

```
336  def require_role(token: str, allowed_roles: List[str]) -> Dict:
337      """
338      Valida el token y verifica que el usuario tenga
339      uno de los roles autorizados.
340      """
341
342      payload = validate_token(token)
343
344      role = payload["role"]
345
346      if role not in allowed_roles:
347          raise PermissionDeniedError(
348              f"Acceso denegado para el rol '{role}'."
349          )
350
351      return payload
352      """
```

Corrección aplicada en la función require_role() para validar los permisos según el rol del usuario. Elaboración propia.

9. Validación posterior a la corrección

```
----------------------------------
Ran 15 tests in 0.612s

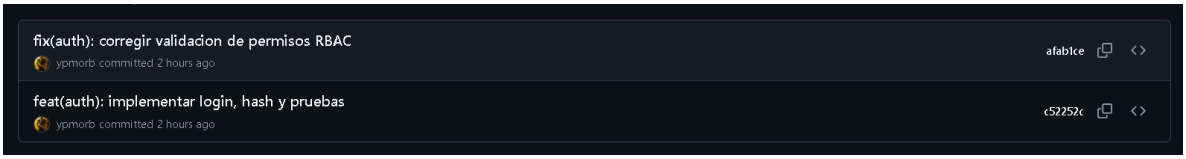
OK
PS C:\Users\yerko\FlotaSegura>
```

```
test_conductor_no_puede_gestionar_mantenciones (test_auth.TestAuthFlotaSegura.test_conductor_n
o_puede_gestionar_mantenciones) ... ok
test_gerente_no_puede_gestionar_mantenciones (test_auth.TestAuthFlotaSegura.test_gerente_no_pu
ede_gestionar_mantenciones) ... ok
```

Ejecución de las pruebas posterior a la corrección del control de acceso, obteniendo 15 casos de prueba satisfactorios. Elaboración propia.

Luego de aplicar la corrección, se ejecutó nuevamente el conjunto completo de 15 pruebas automatizadas. Todos los casos finalizaron satisfactoriamente, incluyendo las pruebas que inicialmente habían detectado la vulnerabilidad de permisos. Esto permitió confirmar que el acceso a las funciones del prototipo se restringe correctamente según el rol del usuario.

10. Evidencia en el control de versiones



Historial de commits del módulo de autenticación, mostrando la implementación inicial y la posterior corrección del control de acceso RBAC. Elaboración propia.

El historial del repositorio permite verificar la trazabilidad del desarrollo, ya que registra tanto la implementación inicial del módulo como la corrección aplicada después de detectar la falla durante las pruebas.

10.Tabla de aporte

Actividad realizada	Descripción del aporte	Evidencia
Desarrollo del módulo de autenticación	Implementación del archivo auth.py para gestionar autenticación y autorización de usuarios en FlotaSegura.	Código auth.py disponible en el repositorio GitHub.
Protección de contraseñas	Implementación de generación y verificación de contraseñas	Funciones hash_password() y verify_password() y

Actividad realizada	Descripción del aporte	Evidencia
	mediante hash, evitando el almacenamiento de claves en texto plano.	pruebas CP-01, CP-02, CP-03 y CP-15.
Inicio de sesión	Implementación de validación de usuario y contraseña mediante la función login().	Evidencia de ejecución exitosa del login y pruebas CP-04 a CP-07.
Generación y validación de token	Implementación de tokens firmados para identificar al usuario autenticado, su rol y controlar su tiempo de vigencia.	Evidencia de token generado y pruebas CP-08, CP-09 y CP-10.
Control de acceso RBAC	Implementación de autorización de funciones según los roles conductor, jefe_flota y gerente.	Función require_role() y pruebas CP-11 a CP-14.
Desarrollo de pruebas automatizadas	Creación del archivo test_auth.py con 15 casos de prueba para validar autenticación, tokens, contraseñas y permisos.	Ejecución python -m unittest test_auth.py -v con resultado Ran 15 tests – OK.
Identificación de falla	Detección de una falla donde se validaba el token, pero no se verificaba correctamente el rol autorizado para una operación.	Evidencia de ejecución con FAILED (failures=2).
Corrección de falla	Modificación de require_role() para comprobar explícitamente el rol antes de autorizar una operación.	Evidencia posterior con las 15 pruebas exitosas.
Control de versiones	Registro de la implementación y de la corrección mediante Git y GitHub.	Commits feat(auth): implementar login, hash y pruebas y fix(auth): corregir validacion de permisos RBAC.

Actividad realizada	Descripción del aporte	Evidencia
Documentación técnica	Documentación del funcionamiento, plan de pruebas, falla detectada y solución implementada.	Secciones correspondientes del informe técnico.